

SYNTAX DIRECTED TRANSLATION

classmate

Date 06/5/2019

Page 69

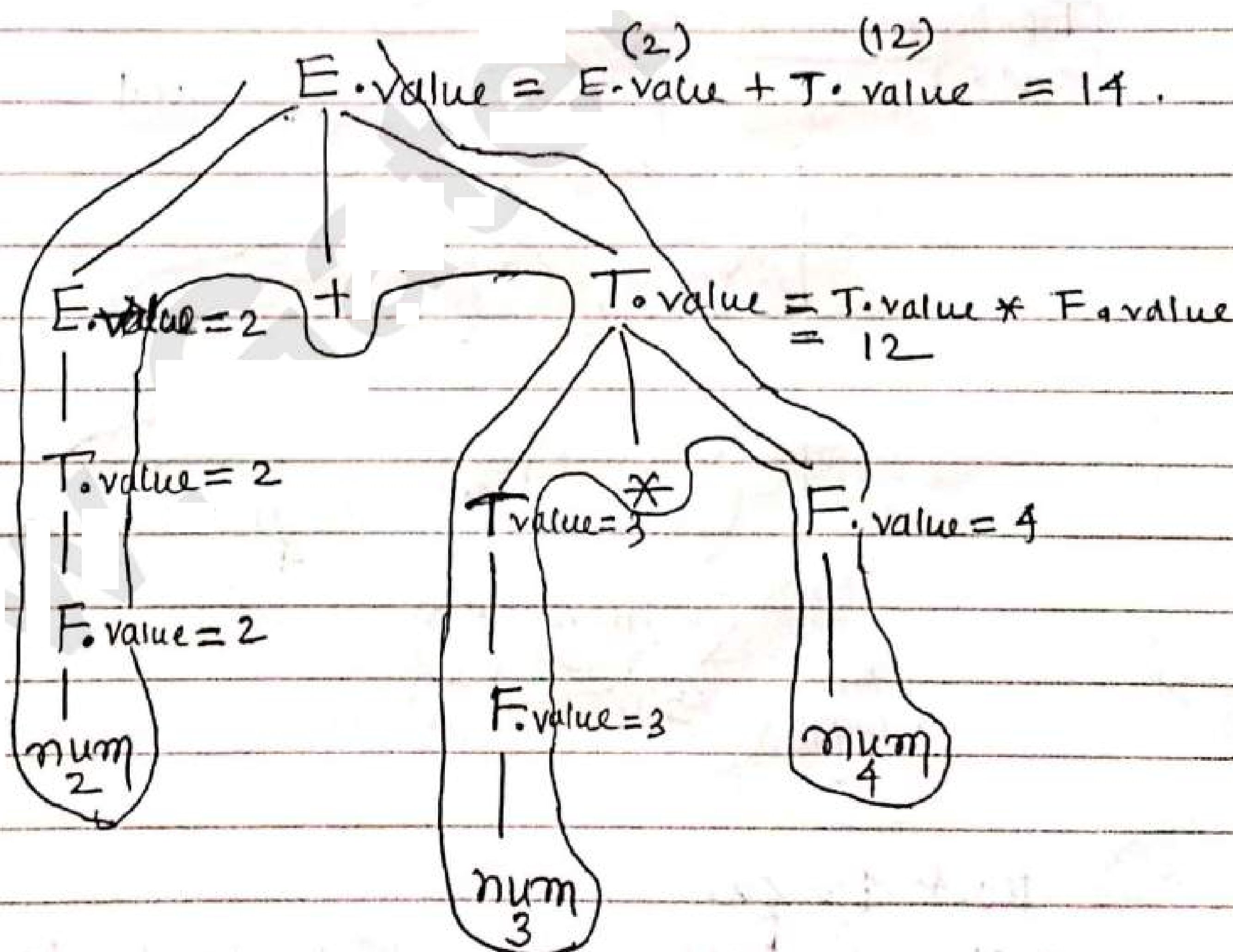
[Grammar + Semantic rules = SDT] (Syntax directed Translation)

STD for evaluation of expression $\Rightarrow$

- ① $E \xrightarrow{\text{grammar}} E + T \xrightarrow{\text{rules}} \begin{cases} E.\text{value} = E.\text{value} + T.\text{value} \\ / T \quad \{ E.\text{value} = T.\text{value} \} \\ T \rightarrow T * F \quad \{ T.\text{value} = T.\text{value} * F.\text{value} \} \\ / F \quad \{ T.\text{value} = F.\text{value} \} \\ F \rightarrow \text{num} \quad \{ F.\text{value} = \text{num}.\text{value} \} \end{cases}$

[ex] =

$2 + 3 * 4$



$\rightarrow$ When reduction will occur, then take production and take action.

②

Symbolic action ①

$E \rightarrow E + T \{ \text{print}("+"); \}$

$/T \{ \}$ ②

$T \rightarrow T * F \{ \text{print}("*"); \}$ ③

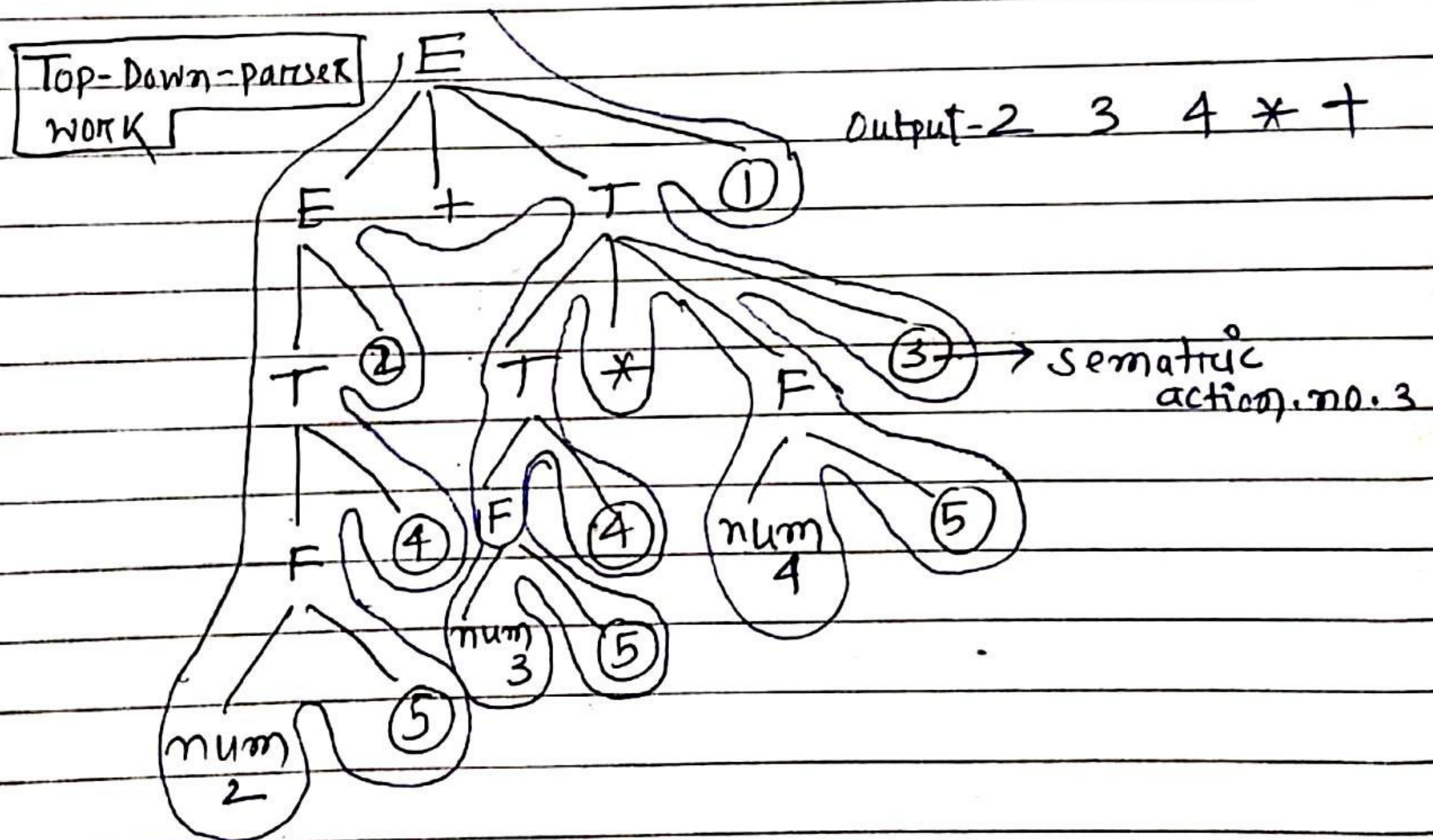
~~redundant~~

$/F \{ \}$ ④

$F \rightarrow \text{num} \{ \text{print}(\text{num.val}); \}$ ⑤

• Top down parser and Bottom-up-parser work with SDT =

ex $\Rightarrow$ ~~2~~ 2 + 3 * 4

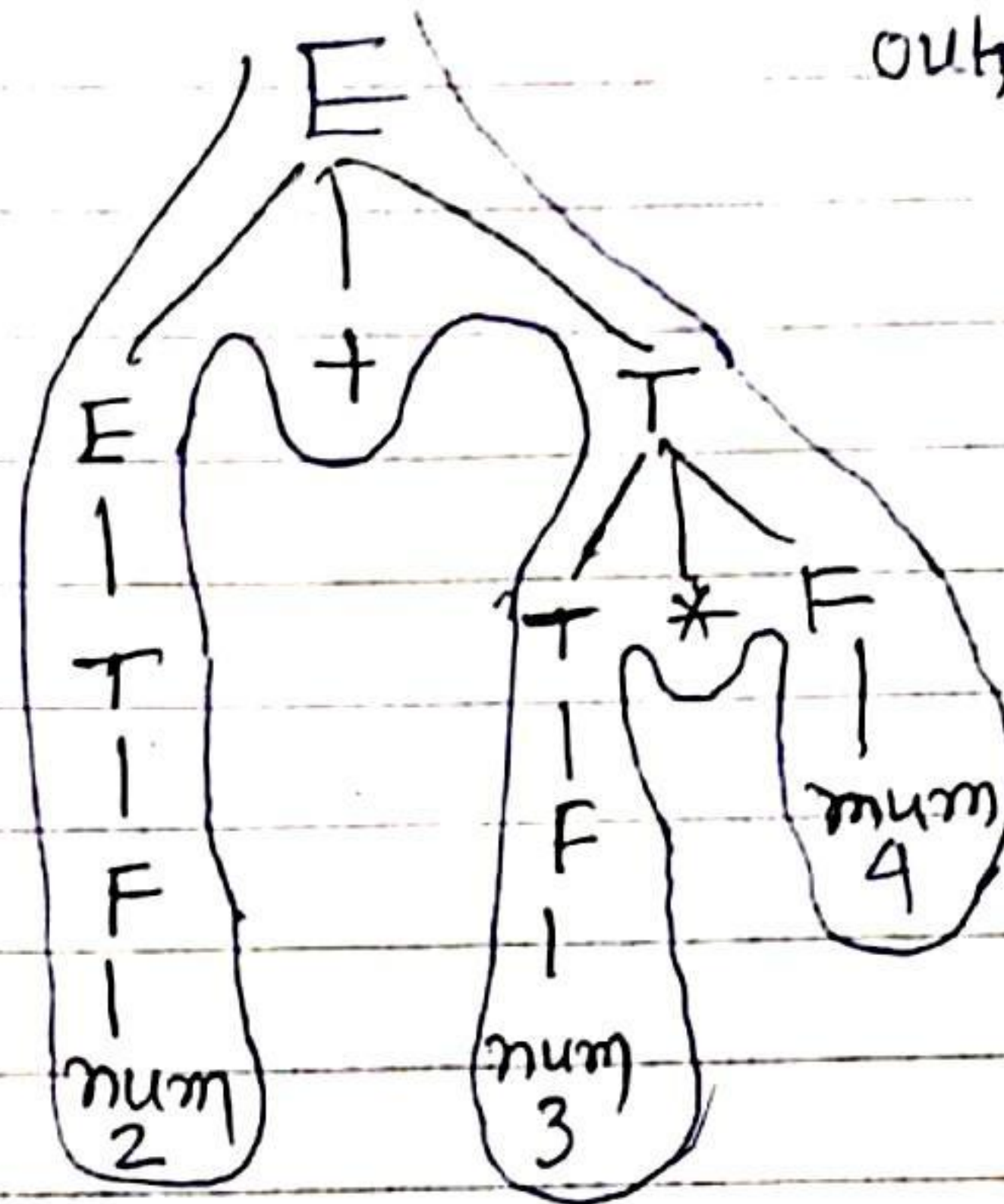


test to do

$\rightarrow$ Top to Bottom and Right left to Right. When see any semantic number then take action.

Bottom-Down-parser
work

output - 2 3 4 * +



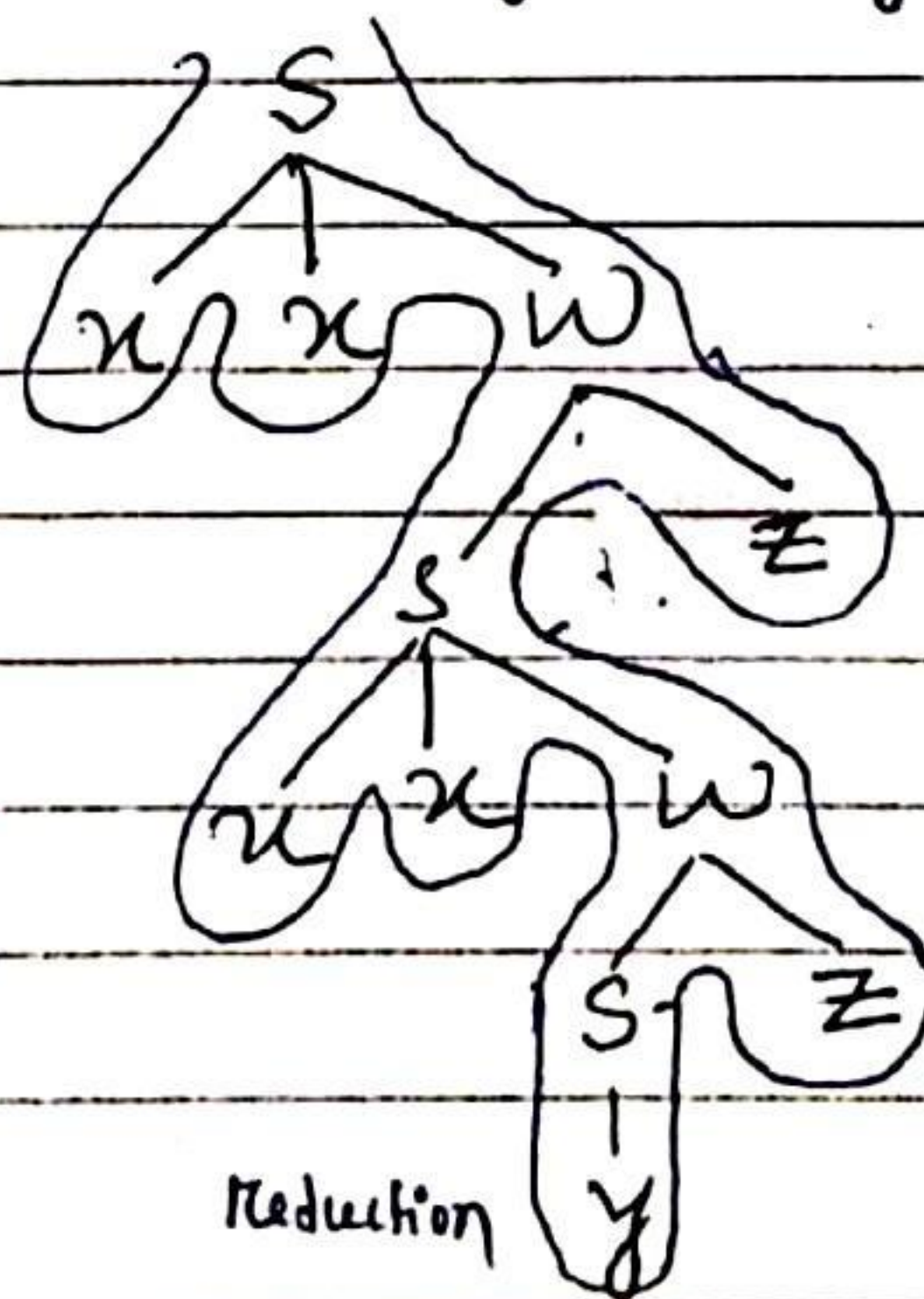
→ using TDP and BUP we get same output.

③

$S \rightarrow x x w \quad \{ \text{printf}(1); \}$
 $\quad \quad \quad / y \quad \quad \{ \text{printf}(2); \}$
 $w \rightarrow s z \quad \quad \{ \text{printf}(3); \}$

String $x x x x y z z$.

generate given string using Bottom Down parser —



output - 2 3 1 3 1

Reduction

④

$$W = 4 - 2 - 4 \times 2$$

$$E \rightarrow E * T \quad \{ E.val = E.val * T.val ; \}$$

$$/ T \quad \{ E.val = T.val ; \}$$

$$T \rightarrow F - T \quad \{ T.val = F.val - T.val ; \}$$

$$/ F \quad \{ T.val = F.val ; \}$$

$$F \rightarrow 2 \quad \{ F.val = 2 ; \}$$

$$/ 4 \quad \{ F.val = 4 ; \}$$

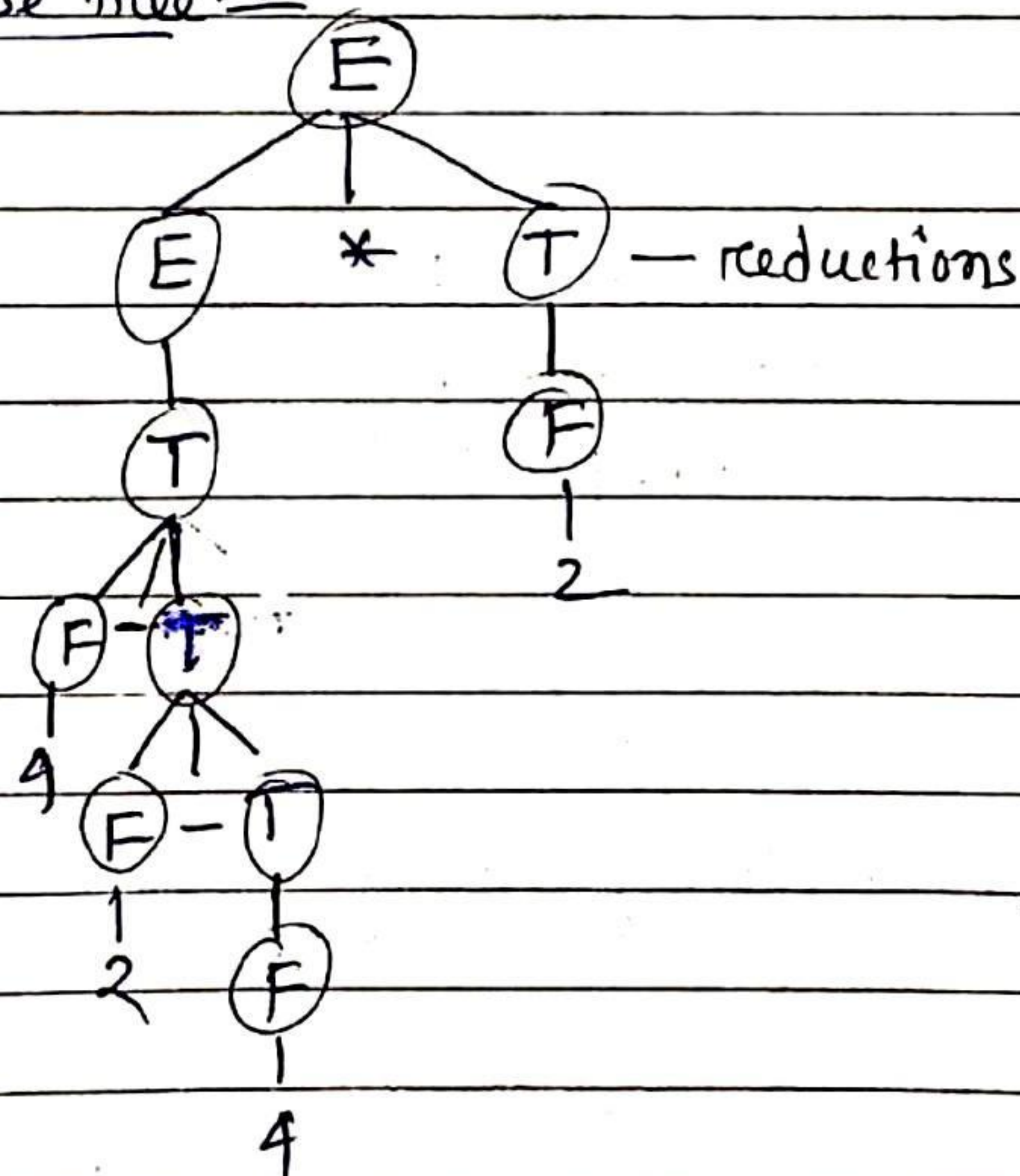
→ In this grammar $- \rightarrow *$.

'*' is left associative.

'-' is Right associative.

———— (Shortcut rule) $W = (4 - (2 - 4) \times 2)$
by not drawing parse tree. = 12

parse tree =



→ Total number of reduction is - 10.

⑤

Semantic action.

$E \rightarrow E \# T$	$\{ E.val = E.val \# T.val; \}$
$/T$	$\{ E.val = T.val; \}$
$T \rightarrow T \& F$	$\{ T.val = T.val + F.val; \}$
Recursion $/F$	$\{ T.val = F.val; \}$
$F \rightarrow num$	$\{ F.val = num.val; \}$

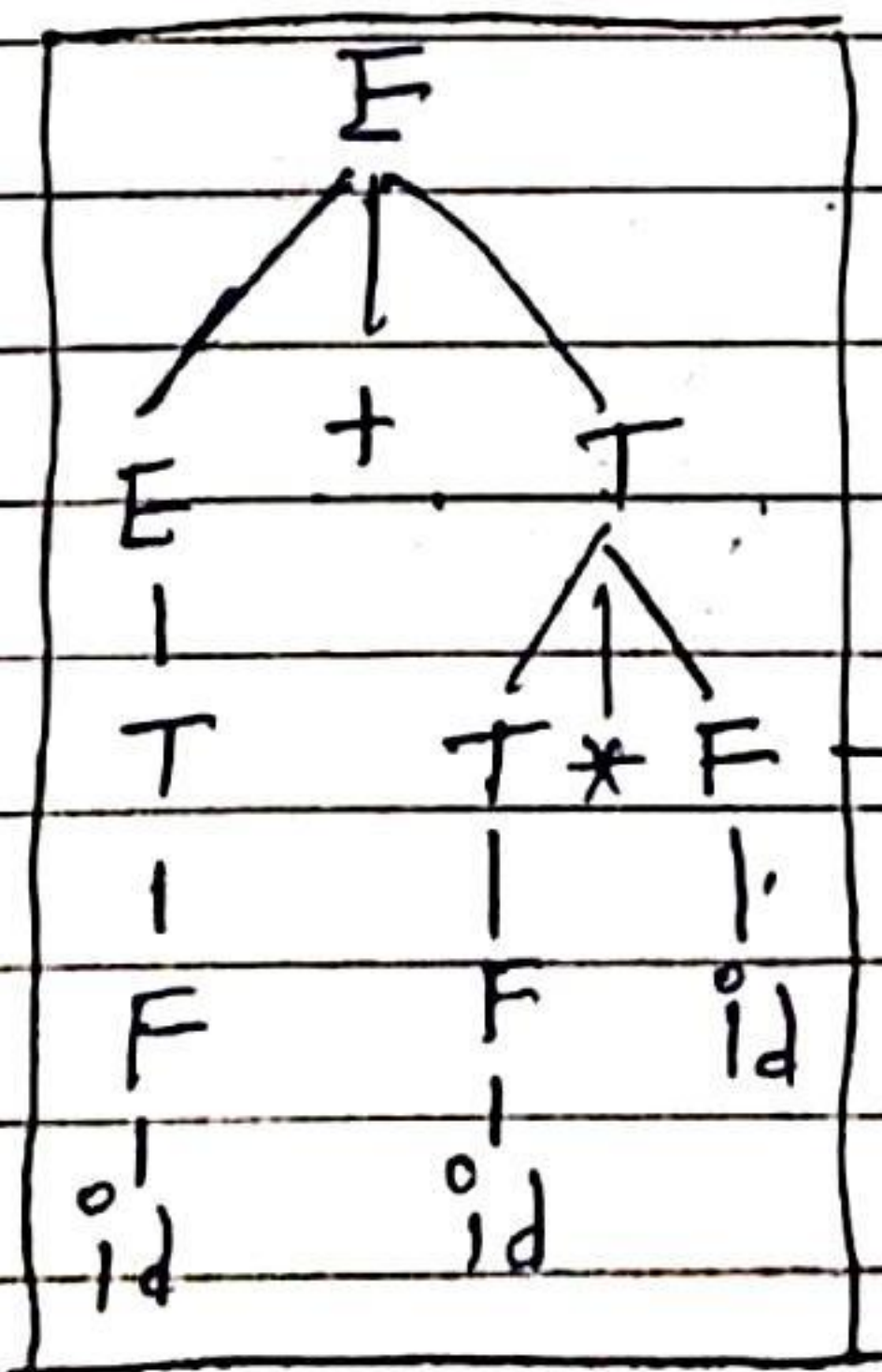
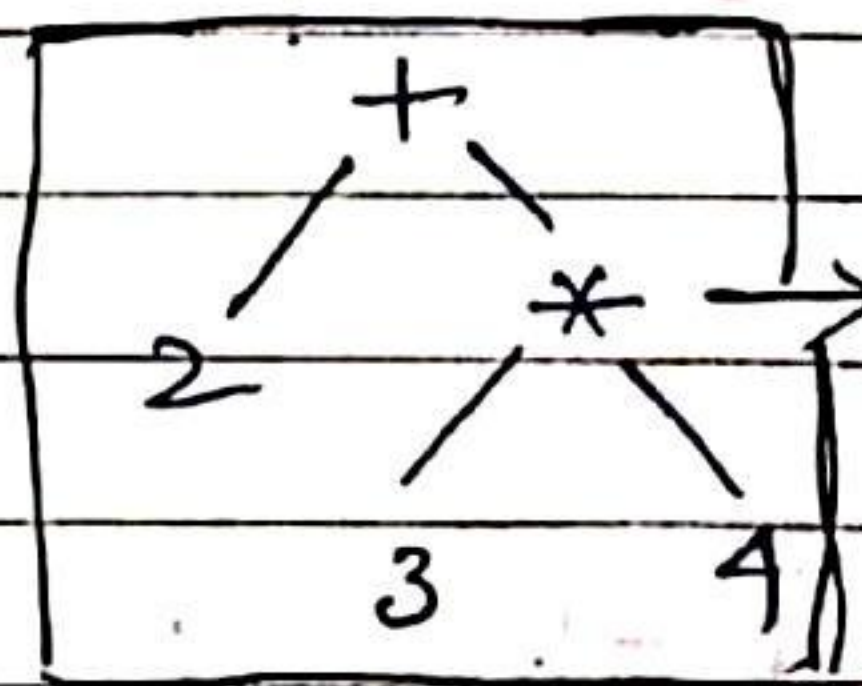
 $W = [2 \# 3 \& 5 \# 6 \& 4] \rightarrow \text{input.}$

$\rightarrow$ in given grammar $(+ \rightarrow *)$, '+' has higher precedence.

$$\begin{aligned}
 & 2 * (3 + 5) * (6 + 4) \\
 &= ((2 * 8) * 10) \\
 &= 160 \text{ (output) (quick way)}
 \end{aligned}$$

$\rightarrow$ Other wise draw parse tree, then find output.

• Abstract and concrete parse tree -

~~Ab con~~ $2 + 3 * 4$  $\rightarrow$ Concrete parse tree.

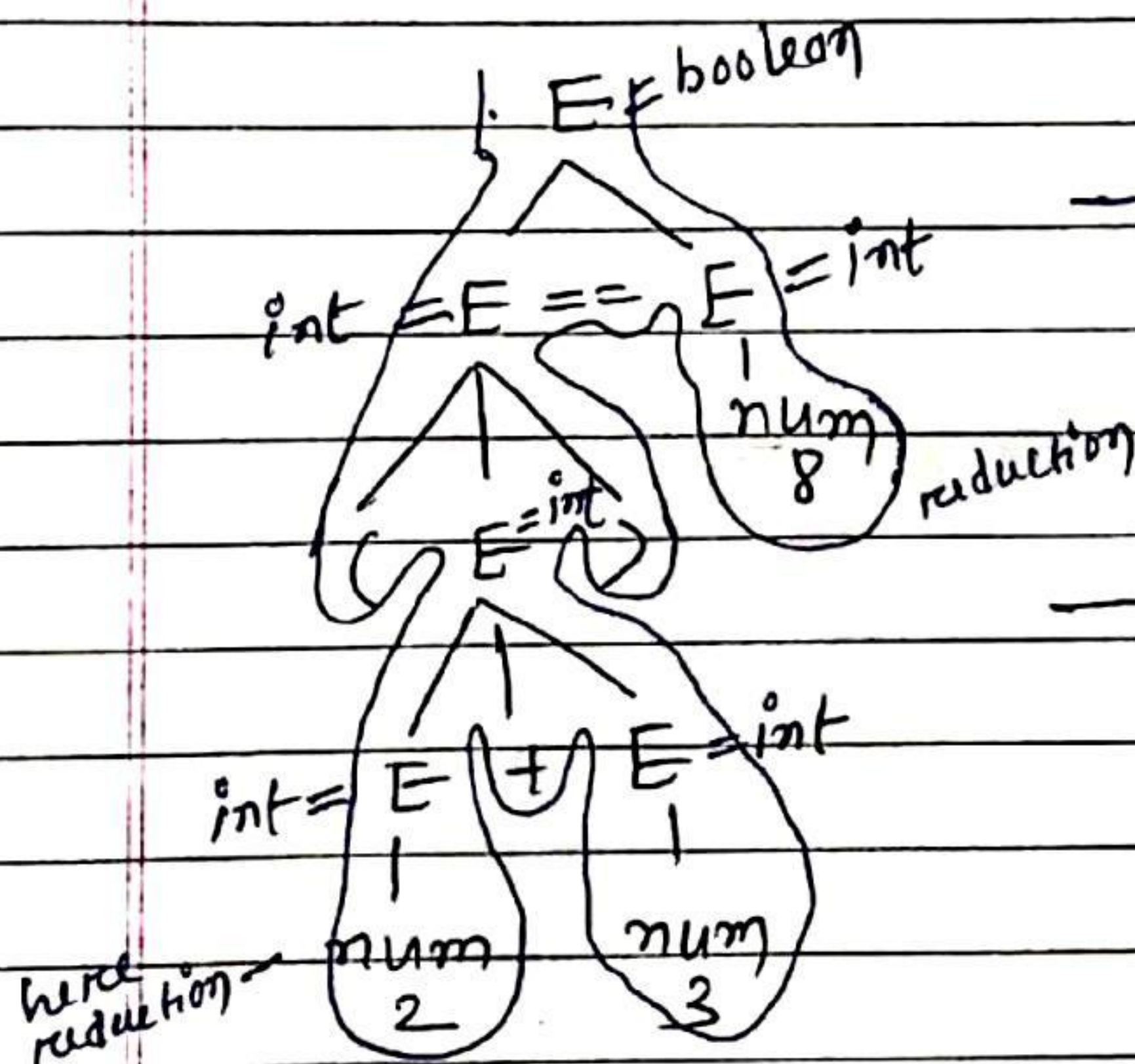
Abstract parse tree.

(variables are not shown)

• STD for type checking =

- ① $E \rightarrow E_1 + E_2 \{ \text{if } ((E_1.\text{type} == E_2.\text{type}) \&\& (E_1.\text{type} == \text{int})) \text{ then } E.\text{type} = \text{int} \text{ else error; } \}$
- $E_1 / E_2 \{ \text{if } ((E_1.\text{type} == E_2.\text{type}) \&\& (E_1.\text{type} == \text{int}/\text{boolean})) \text{ then } E.\text{type} = \text{boolean} \text{ else error; } \}$
- $/ (E_1) \{ E.\text{type} = E_1.\text{type}; \}$
- $/ \text{num} \{ E.\text{type} = \text{int}; \}$
- $/ \text{True} \{ E.\text{type} = \text{bool}; \}$
- $/ \text{False} \{ E.\text{type} = \text{bool}; \}$

Ex = $(2+3) == 8$ (find the type of expression)



→ according to give rules.

→ So the given expression is Boolean expression.

- Given a string, find out no. of 1's in string using this grammar (using a SDT).

	count 1's	count 0's	count bit
$N \rightarrow L$	$\{N.C = L.C\}$	.	
$L \rightarrow LB$	$\{L.C = L.C + B.C\}$		
$/ B$	$\{L.C = B.C\}$		
$B \rightarrow 0$	$\{B.C = 0\}$	$B.C = 1$	$B.C = 1$
$/ 1$	$\{B.C = 1\}$	$B.C = 0$	$B.C = 1$

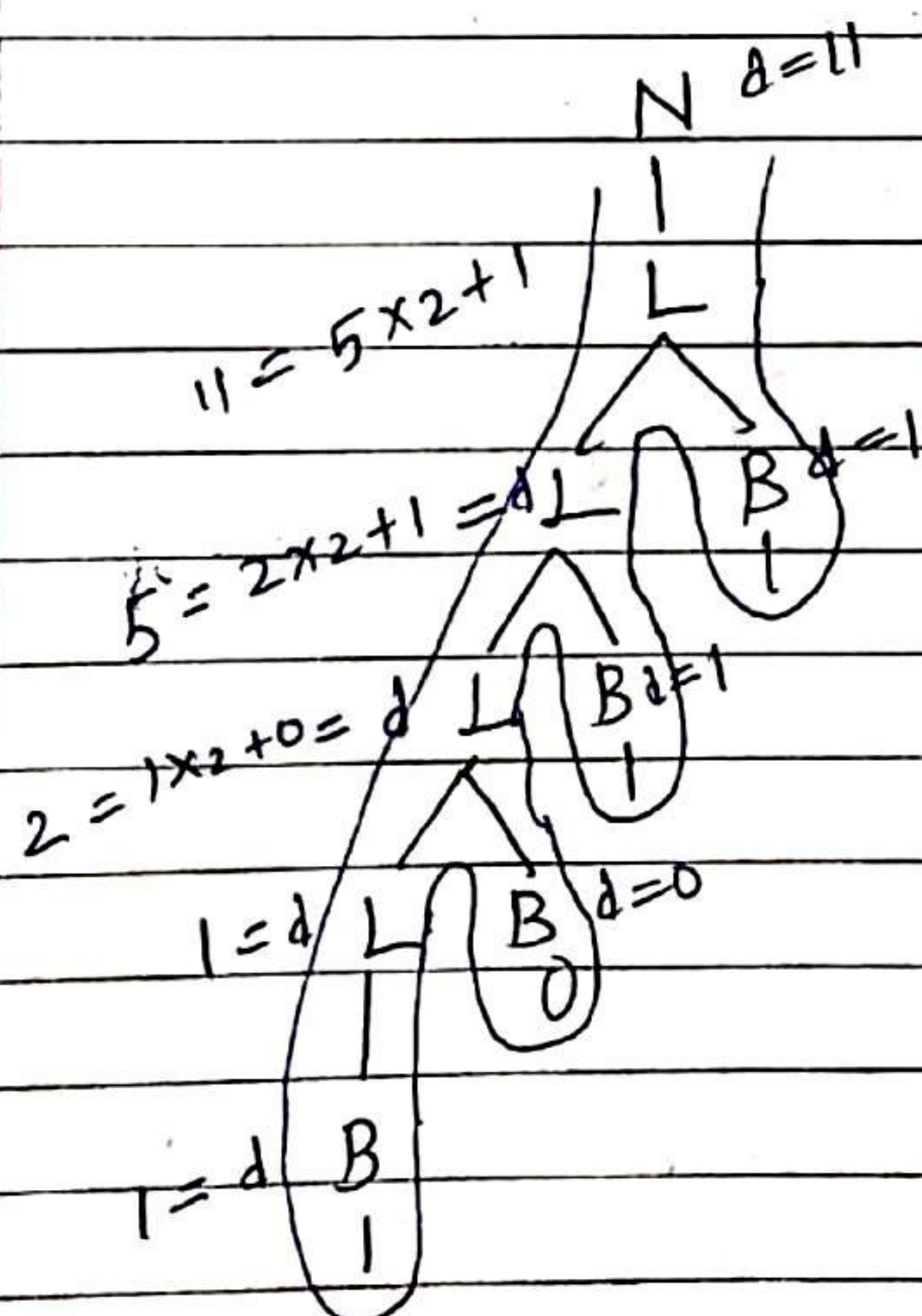
B.C → Bit count

- Convert binary to Decimal using the grammar →

$$\begin{array}{r} \underline{10110} \\ 11 \times 2 + 0 \\ = 22 \end{array} \quad \begin{array}{r} \underline{10111} \\ 11 \times 2 + 1 \\ = 23 \end{array}$$

$N \rightarrow L$	$\{N.dval = L.dval\}$	
$L \rightarrow LB$	$\{L.dval = L.dval \times 2 + B.dval\}$	
$/B$	$\{L.dval = B.dval\}$	
$B \rightarrow 0$	$\{B.dval = 0\}$	(dval → decimal value)
$/1$	$\{B.dval = 1\}$	

Ex = find out the decimal value of given string -
 (SDB) 1011 → decimal value (11).



(d → decimal)

11.01 → you should change the rule to
 [3.25] Convert 11.01 to decimal

• Convert binary number with Decimal point to Decimal grammar =

$$11.01$$

$$\cdot 01 = \frac{1}{2^2} = 0.25$$

$$\boxed{11.01 = 3.025}$$

$$11.11$$

$$\cdot 11 = \frac{3}{2^2} = \frac{3}{4} = 0.75$$

$$\boxed{11.11 = 3.75}$$

S-rule

constr

$$N \rightarrow L_1 L_2 \{ N.dval = L_1.dval + \frac{L_2.dval}{2^{L_2.c}} \}$$

$$L \rightarrow L_1 B \{ L.c = L_1.c + B.c; L.dval = L_1.dval + B.dval; \}$$

$$/ B \{ L.c = B.c; L.dval = B.dval; \}$$

$$B \rightarrow 0 \{ B.c = 1; B.dval = 0 \}$$

$$/ 1 \{ B.c = 0; B.dval = 1 \}$$

• SDT to generate three address code = $T \rightarrow \text{term}$
 $F \rightarrow \text{factor}$

(semantic rule)

$$S \rightarrow id = E \{ gen(id.name = E.place); \}$$

$$E \rightarrow E_1 + T \{ E.place = newTemp(); gen(E.place = E_1.place + T.place); \}$$

$$/ T \{ E.place = T.place; \}$$

$$T \rightarrow T * F \{ T.place = newTemp(); gen(T.place = T_1.place * F.place); \}$$

$$/ F \{ T.place = F.place; \}$$

$$F \rightarrow id \{ F.place = id.name; \}$$

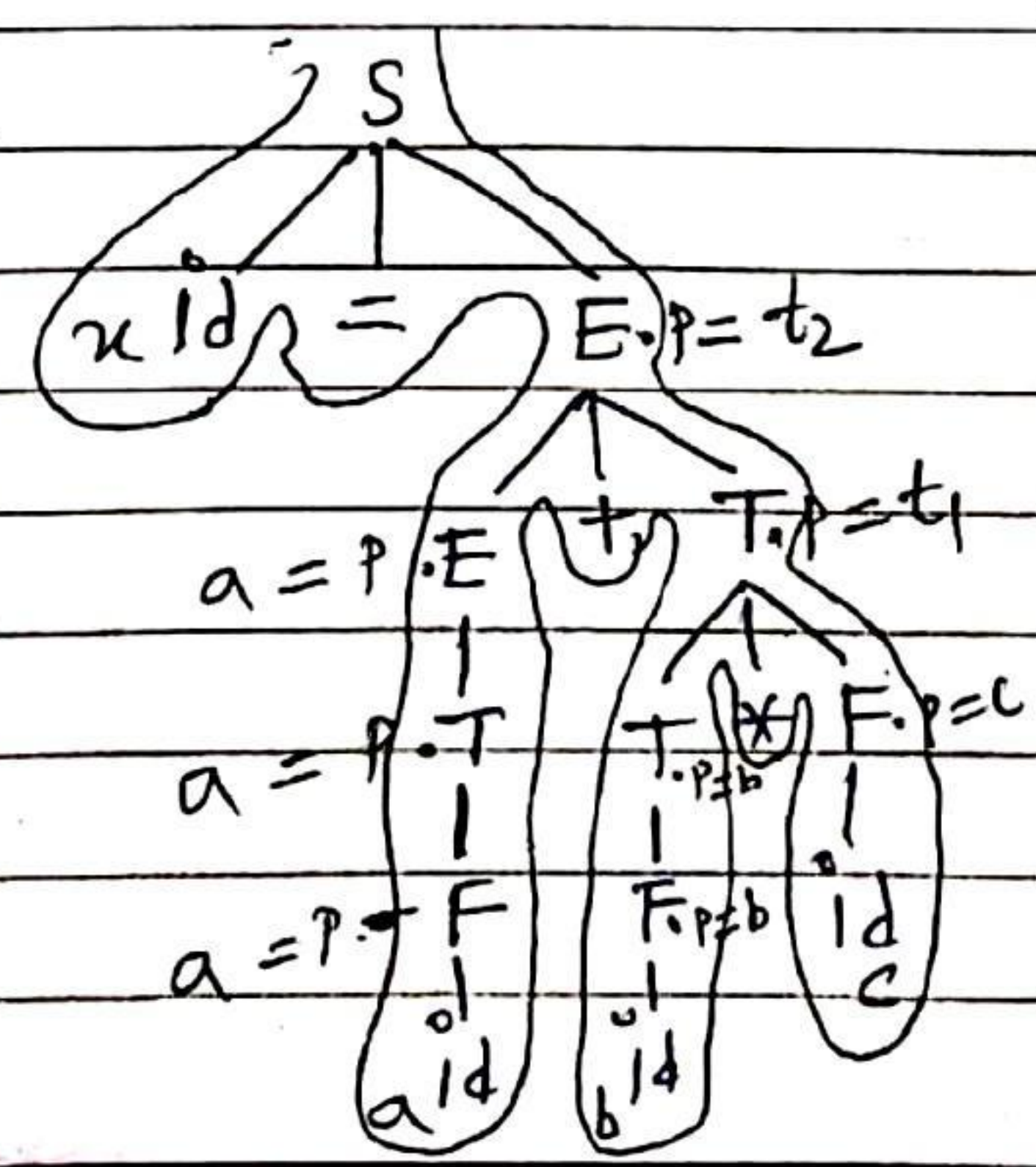
ex =

$$\boxed{x = a + a * c}$$

output:

$$\begin{aligned} t_1 &= b * c \\ t_2 &= a + t_1 \\ x &= t_2 \end{aligned}$$

Bottom of parsing



- SDT are two types =

(i) S-attribute SDT.

(ii) L-attribute SDT.

$A \rightarrow BCD$
 $A.s \rightarrow f(B.s, C.s, D.s)$
 ('A' is taking value from its children)

→ synthesized attributes.

$A \rightarrow BCD$
 $C.i \rightarrow A.i$
 $E.i \rightarrow B.i$
 $C.i \rightarrow D.i$

→ if 'C' taking attribute from its parents and siblings, it called inherited attribute

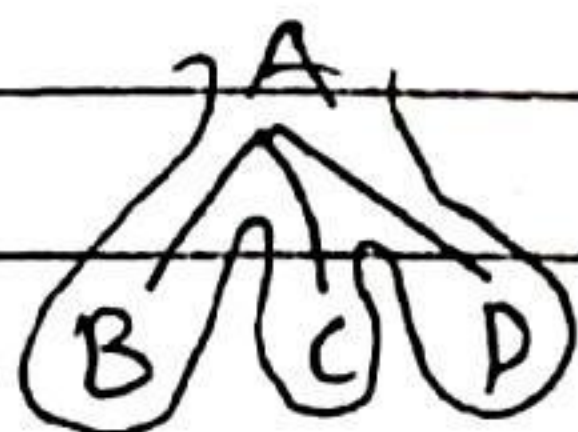
S-attribute SDT

1) Uses only synthesized attributes

2) semantic ~~attributions~~ actions are placed at right end of production.

$A \rightarrow BCC\{\}$

3) Attributes are evaluated during Bottom-Down-parsing.



L-attribute SDT

1) Uses both inherited and synthesized attributes. Each inherited attribute is restricted to inherit either from ~~parent~~ parent or left siblings only.

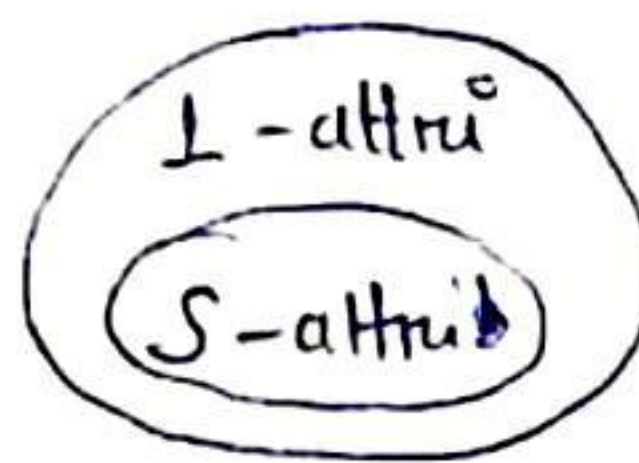
Ex: $A \rightarrow XYZ \begin{cases} Y.s = A.s \checkmark \\ Y.s = X.s \checkmark \\ Y.s = Z.s \end{cases}$

2) semantic actions are place anywhere on R.H.S.

3) Attributes are evaluated by traversing parse tree depth first, left to right.



Attribute = value



Example — find out SDT are S-attributed or L-attributed =

- ① $A \rightarrow LM \{ \overset{\text{Inherited}}{L.i = f(A.i)}; \overset{\text{not S-att}}{M.i = f(L.s)}; \overset{\text{not L-att}}{A.s = f(M.s)}; \}$
 $A \rightarrow QR \{ R.i = f(A.i); \overset{\text{not L-att}}{Q.i = f(R.i)}; A.s = f(Q.s); \}$

- Ⓐ S-attributed Ⓒ both
 Ⓑ L-attributed Ⓓ none

- ② $A \rightarrow BC \{ \overset{\text{not S-attri}}{B.s = A.s}; \}$

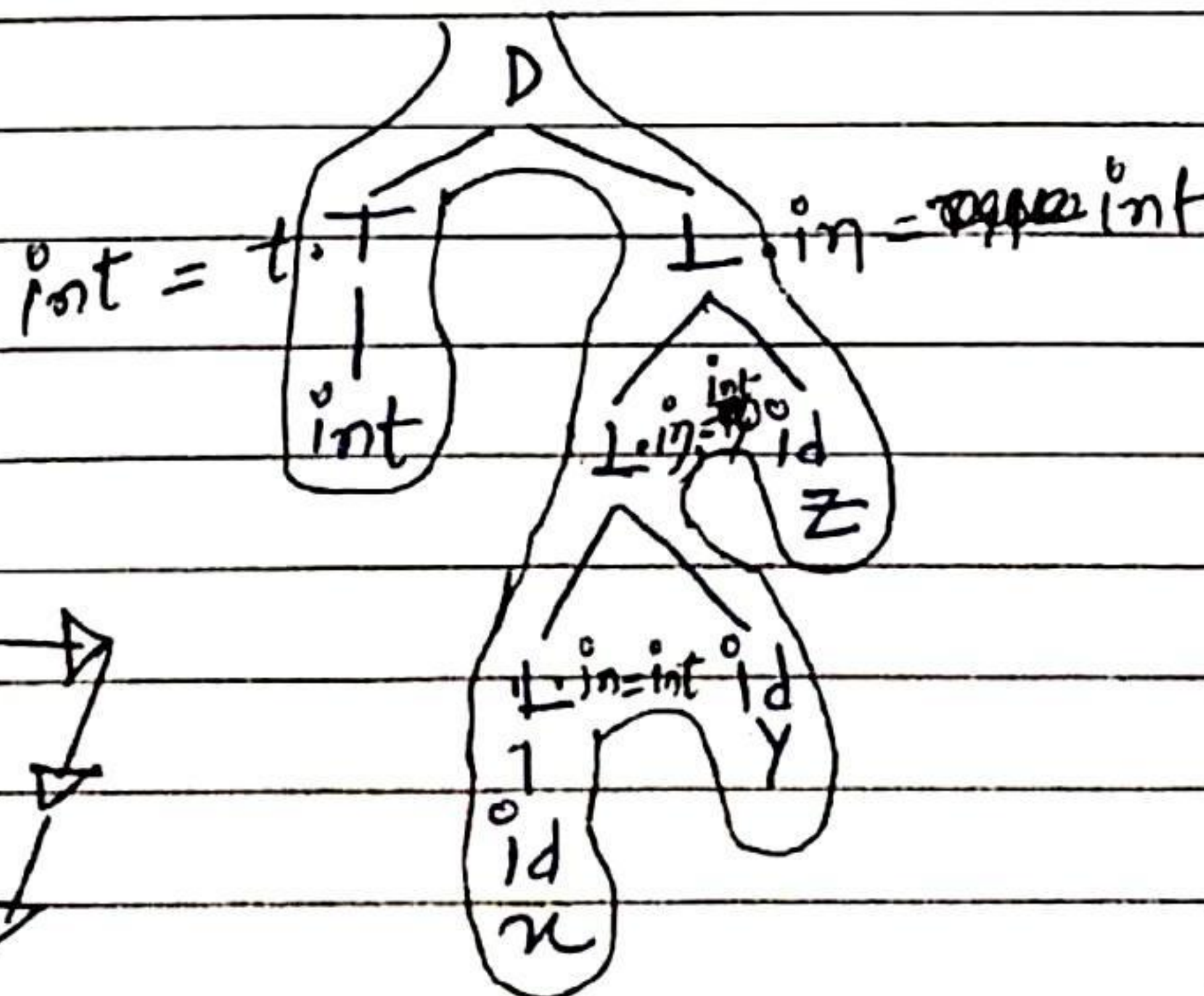
- Ⓐ S-attri Ⓒ both
 Ⓑ L-attri Ⓓ none

• SDT to store type information into symbol table —

- ① grammar + rule = SDT
- $D \rightarrow TL \{ L.in = T.type \}$ L-attribute
 $T \rightarrow \text{int} \{ T.type = \text{int}; \}$
 $\quad \quad \quad / \text{char} \{ T.type = \text{char}; \}$
 $L \rightarrow L, id \{ L.in = L.in, \text{add type}(id.name, L.in); \}$
 $\quad \quad \quad / id \{ \text{add type}(id.name, L.in); \}$

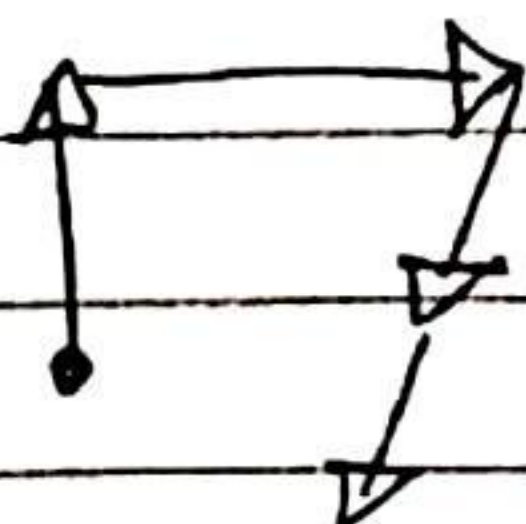
→ (this SDT are L-attributed definition) int x, y, z

ex =



id name	
x	int
y	int
z	int

T → type



②

S-attribute

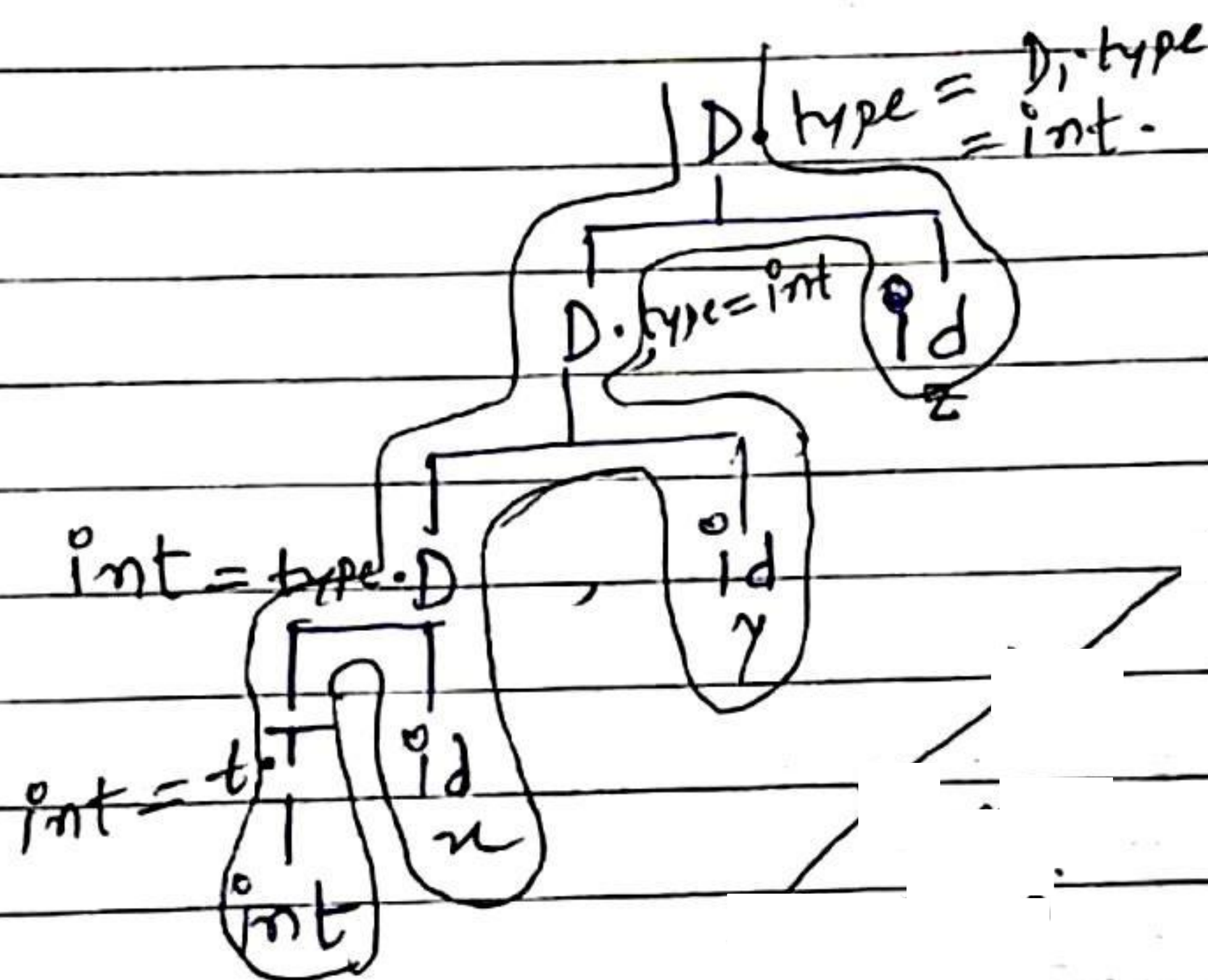
$D \rightarrow D_1, id \{ \text{add-type}(id.name, D_1.type) \}; D.type = D_1.type \}$

$/ T id \{ \text{add-type}(id.name, T.type), D.type = T.type \}$

$T \rightarrow int \{ T.type = int \}$

$/ char \{ T.type = char \}$

Ex = int x, y, z



x	int
y	int
z	int